

NEXA SaaS

MACH microservices migratieplan

Een gefaseerd plan om de huidige modulaire Laravel SaaS-applicatie om te zetten naar een MACH-architectuur met microservices, API-first contracten, cloud-native containers en headless clients.

Datum: 2026-06-03

NEXA SaaS MACH microservices migratieplan

Datum: 2026-06-03

Scope: huidige NEXA SaaS codebase met Core, Nexa Taxi, Nexa Skillmatching, website builder, admin, chauffeur app en toekomstige modules.

1. Executive summary

NEXA is nu een modulaire Laravel SaaS-applicatie. De codebase heeft al belangrijke bouwstenen voor een latere microservices-opzet:

- Modules registreren eigen menu-items, permissies, routes, migraties en frontend componenten.
- Er is al een module registry via `modules` en tenant-koppeling via `company\_module`.
- Er is al database-isolatie via `MODULE\_DATABASE\_STRATEGY=single|schema|database`.
- Nexa Taxi heeft al headless API's voor de chauffeur app.
- Website builder en frontend componenten zijn al losser gekoppeld aan modules.

Het beste migratiepad is geen big bang rewrite. De aanbevolen route:

1. Stabiliseer de bestaande applicatie als modular monolith met harde modulegrenzen.
2. Introduceer API Gateway, service contracts, event outbox en tenant context.
3. Splits eerst de meest zelfstandige domeinen af: Taxi Dispatch/Driver API, Notifications, Billing/Payments.
4. Splits daarna Skillmatching matching engine en eventueel public website rendering.
5. Laat Core/IAM, tenantbeheer, module registry en admin shell langer centraal, totdat contracten stabiel zijn.

Doelarchitectuur volgens MACH:

- Microservices: domeinservices met eigen code, eigen datamodel en eigen deploy.
- API-first: alle functies via versioned REST/JSON of GraphQL contracten, plus webhooks/events.
- Cloud-native: containers voor services, managed PostgreSQL/Redis/object storage in productie.
- Headless: admin UI, tenant websites, chauffeur app en toekomstige apps consumeren APIs.

2. Huidige situatie en uitgangspunten

Huidige domeinen

- Core platform: bedrijven, users, rollen, permissies, modules, tenant settings, configuraties.
- Admin backend: Metronic/Laravel admin voor beheer, menu's, rollen en moduleconfiguratie.
- Website builder: `website\_pages`, `frontend\_themes`, `website\_media`, component registry.
- Nexa Taxi: voertuigen, tarieven, ritten, dispatch, chauffeur API, betalingen en facturen.
- Nexa Skillmatching: vacatures, kandidaten, matches, interviews, branches, frontend portaal.
- Shared services: e-mailtemplates, notificaties, chat, invoices, payments, tenant sync.

Belangrijke ontwerpkeuze

Voor microservices mag een service niet rechtstreeks tabellen van een andere service lezen of schrijven. Integratie loopt via:

- API calls voor synchrone vragen of commands.
- Events voor asynchrone updates.
- Read models voor overzichten en dashboards.
- Webhooks voor externe koppelingen.

Gefaseerd uitgangspunt

De huidige `schema` strategie is een goede tussenstap:

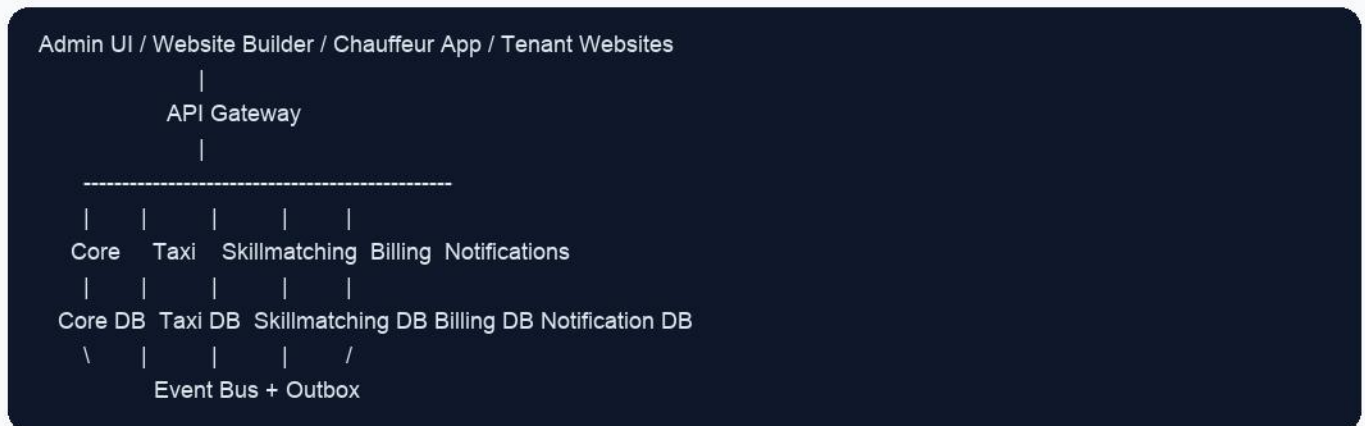
- Een PostgreSQL database.
- `public` schema voor core.
- `nexa\_taxi`, `nexa\_skillmatching`, enz. als module-schema's.

Voor echte microservices wordt dit later:

- Een database of schema per service.
- Geen cross-service joins.
- Data-eigenaarschap per service.

3. MACH doelarchitectuur

Conceptueel doelbeeld



Microservices

Services worden gesplitst op business capability, niet op technische laag. Een module kan een service worden, maar sommige modules bestaan beter uit meerdere services wanneer de runtime-eisen verschillen. Taxi dispatch is bijvoorbeeld realtime/headless en hoort uiteindelijk losser te draaien dan Taxi admin.

API-first

Elke service krijgt:

- OpenAPI specificatie per versie, bijvoorbeeld `/api/v1/taxi/rides``.
- Strikte request/response DTO's.
- Service-to-service auth.
- Idempotency keys voor betaal-, factuur- en dispatch-acties.
- Webhooks voor externe systemen.

Cloud-native

Services draaien als containers. Data draait in productie bij voorkeur niet in containers, maar in managed databases, queues en object storage.

Headless

Alle frontends worden clients:

- Admin shell gebruikt Core, Module, Taxi, Skillmatching APIs.
- Website builder publiceert headless pagina-content/componenten.
- Chauffeur app gebruikt Taxi Driver API.
- Toekomstige mobiele apps gebruiken dezelfde gateway/contracts.

4. Voorgestelde service boundaries

Service	Verantwoordelijkheid	Startpositie	Target DB
API Gateway / BFF	Routing, rate limiting, auth enforcement, tenant context, request logging	Nieuw toevoegen voor alle clients	Geen eigen business DB
Identity & Access	Users, rollen, permissies, login, tokens, MFA, service accounts	Eerst in Core laten	Eigen IAM DB of managed IdP
Tenant & Module Registry	Companies, modules, company_module, module config, abonnementen	Core blijft eigenaar	Core DB
Admin Shell	UI container voor backend admin en module menu's	Laravel/Blade blijft eerst	Geen DB, alleen APIs
Website Builder	Pagina's, thema's, media, component registry, public page API	Later los trekken	Website DB + object storage
Taxi Booking	Ritverzoeken, offertes, route, voertuigen, tarieven	Nexa Taxi module	Taxi DB
Taxi Dispatch / Driver API	Chauffeur availability, offers, realtime stream, start/complete ride	Eerste kandidaat voor extractie	Taxi Dispatch DB of Taxi DB fase 1
Billing & Payments	Payment providers, Mollie, ride_payments, payments, invoices, PDF facturen	Los trekken na Taxi API	Billing DB
Notifications	E-mail, SMS, templates, notification logs, retry queue	Los trekken vroeg	Notification DB
Skillmatching ATS	Vacatures, kandidaten, interviews, branches, pipeline	Skillmatching module	Skillmatching DB
Matching Engine	Matchscores, AI analyse, embeddings, search/vector index	Los van ATS zodra AI groeit	Matching DB + vector store
Chat	Chat rooms, messages, participants, presence	Apart bij schaal/realtime	Chat DB
Reporting / Analytics	Dashboards, KPI's, exports, audit/report read models	Later toevoegen	Analytics warehouse/read DB

5. Databasekeuze per domein

Hoofdadvies

Gebruik in productie containers voor applicaties, maar geen database-in-container als primaire opslag. Gebruik een vaste managed PostgreSQL database of cluster met backups, PITR, monitoring en encryption at rest.

Voor development en test:

- Docker Compose met Postgres, Redis, Mailpit en services is prima.
- Seed/demo data kan per service of module apart.

Voor productie:

- Services in containers.
- Databases managed of vaste dedicated database server.
- Per service een eigen database of minimaal eigen schema.
- Geen directe databasekoppeling tussen services.

Database-indeling

Domein	Gezamenlijk of los	Advies
Core tenant/module registry	Gezamenlijk platform DB	Houd centraal. Dit is de bron voor companies, modules en tenant subscriptions.
IAM/RBAC	Los zodra services volwassen zijn	Eerst Core DB; later eigen IAM DB of externe IdP zoals Keycloak/Auth0/Entra.
Website Builder	Losse DB of schema	Eigen data-eigenaar voor pages/themes/media. Media in object storage.
Taxi Booking	Losse DB/schema	Eigen service DB. Ritten, voertuigen, tarieven en booking payloads horen hier.
Taxi Dispatch	Fase 1 in Taxi DB, later los	Bij realtime schaal eigen DB/read model voor availability/offers.
Billing/Payments	Losse DB	Sterk afgeschermd domein. Geen andere service schrijft payment/factuur-tabellen.
Notifications	Losse DB	Eigen retry status, logs, templates en delivery state.
Skillmatching ATS	Losse DB/schema	Vacatures, kandidaten, interviews, pipeline en branches.
Matching Engine/AI	Losse DB + vector index	Embeddings, scores, AI analyses en zoekindex scheiden van ATS transactiedata.
Analytics	Losse read database/warehouse	Events kopiëren naar analytics; niet rechtstreeks productiedata joinen.

Wanneer wel gezamenlijk?

Alleen voor de overgangsfase mag een gedeelde PostgreSQL cluster gebruikt worden. Dan gelden deze regels:

-

Per service eigen schema of database.

- Database user per service met alleen rechten op eigen schema.
- Geen foreign keys over servicegrenzen.
- Geen cross-schema joins in applicatiecode.
- Integratie via API of events.

6. Containers, vaste database en infrastructuur

Development

Gebruik Docker Compose:

- `api-gateway`
- `core-service`
- `taxi-service`
- `skillmatching-service`
- `billing-service`
- `notifications-service`
- `postgres`
- `redis`
- `mailpit`
- optioneel `minio` voor object storage

Voordeel: elke developer kan het hele platform lokaal starten.

Staging

Gebruik containers plus een vaste staging database:

- Services apart deploybaar.
- 1 PostgreSQL cluster met database/schema per service.
- Redis voor queues/cache.
- Object storage voor media/facturen.
- Separate secrets per service.

Productie

Gebruik cloud-native managed componenten:

- Containers via Kubernetes, ECS, Nomad, Docker Swarm of managed app platform.
- Managed PostgreSQL voor data.
- Managed Redis of queue service.
- S3-compatible object storage.
- Centralized logs, metrics en tracing.
- Backups en restore-tests.

Waarom geen productie-DB in container?

Een database-in-container kan technisch, maar is kwetsbaarder voor:

- volume/config fouten;
- backup discipline;
- failover;
- upgrades;

- monitoring;
- storage performance.

Gebruik containers voor compute, vaste/managed databases voor state.

7. API, headless en integratiepatronen

API Gateway

De API Gateway wordt de enige publieke ingang:

- `/api/v1/core/*`
- `/api/v1/taxi/*`
- `/api/v1/driver/*`
- `/api/v1/skillmatching/*`
- `/api/v1/website/*`
- `/api/v1/billing/*`

Taken:

- tenant resolutie op host/header/token;
- JWT validatie;
- rate limiting;
- request IDs;
- CORS;
- body size limits;
- audit logging;
- version routing.

Headless clients

Client	Benodigde APIs
Admin backend	Core, IAM, Module Registry, Taxi Admin, Skillmatching Admin, Billing
Tenant website	Website Builder API, public component data, Taxi booking, public vacancies
Chauffeur app	Driver auth, availability, dispatch inbox/stream, payments, invoice send
Klant portaal	Auth, eigen ritten, betalingen, facturen, profiel
Kandidaat portaal	Auth, vacatures, matches, favorieten, interviews

Events

Gebruik events om services los te koppelen:

- `RideRequested`
- `RideQuoted`
- `RideAcceptedByDriver`
- `RideCompleted`
- `PaymentPaid`
- `InvoiceSent`
-

`VacancyPublished`

- `CandidateApplied`
- `MatchCalculated`
- `InterviewScheduled`
- `NotificationDeliveryFailed`

Begin pragmatisch met database outbox + queue worker. Later kan dit naar RabbitMQ, NATS, Kafka of cloud pub/sub.

8. Veiligheid en compliance

Auth en autorisatie

- Gebruik OIDC/JWT voor gebruikers en service accounts.
- Houd RBAC centraal: rollen, permissies, tenant en module-entitlements.
- Voeg scopes toe voor API clients, bijvoorbeeld `taxi:rides.write`.
- Gebruik korte token lifetimes en refresh token rotatie.
- Voor service-to-service verkeer: mTLS of signed service tokens.

Tenant-isolatie

- Elke request krijgt een verplichte `tenant\_id` context.
- Services valideren tenant access zelf, niet alleen de gateway.
- Database policies of minimaal `company\_id` filters afdwingen.
- Voor grote klanten kan database-per-tenant later worden toegevoegd.

Data security

- Encryptie at rest en in transit.
- Secrets via vault/secret manager, niet in `.env` in images.
- PII minimaliseren in events; gebruik IDs en snapshots waar nodig.
- Audit log voor admin, betalingen, facturen, ritstatus en sollicitatie-acties.
- Rate limits op publieke booking, login, driver APIs en webhooks.

Betaling en factuur

- Billing service wordt enige eigenaar van Mollie/payment state.
- Webhooks verifiëren met provider signature/secret.
- Payment acties idempotent maken.
- Factuur-PDF's private opslaan in object storage.

9. Migratiefases

Fase 0 - Voorbereiding

- Leg module boundaries vast in code en documentatie.
- Stop nieuwe cross-module model calls waar mogelijk.
- Voeg service DTO's toe voor Taxi, Skillmatching, Billing en Notifications.
- Maak OpenAPI specs voor bestaande public/driver APIs.
- Introduceer request ID, tenant context en audit logging.

Fase 1 - Modular monolith hardening

- Gebruik `MODULE\_DATABASE\_STRATEGY=schema` als standaard.
- Zorg dat module-tabellen alleen in eigen schema staan.
- Maak database users per module-schema.
- Introduceer outbox tabel per domein.
- Maak API endpoints leidend, ook intern.

Fase 2 - Eerste extracties

Splits eerst services die weinig UI-afhankelijkheid hebben:

- Notifications service.
- Billing/Payments service.
- Taxi Dispatch/Driver API.

Deze leveren direct waarde voor headless en veiligheid.

Fase 3 - Taxi service als zelfstandige module

- Taxi Booking API wordt eigenaar van rides, vehicles, rates.
- Driver API draait apart maar gebruikt Taxi events.
- Admin gebruikt Taxi API in plaats van Eloquent model calls.
- Billing wordt alleen via API/events aangeroepen.

Fase 4 - Skillmatching service splitsen

- ATS service beheert vacatures, kandidaten, interviews.
- Matching Engine beheert AI, embeddings en scores.
- Public vacancy pages lezen via Website/Skillmatching API.
- Candidate portal wordt headless client.

Fase 5 - Platform composability

- Nieuwe modules krijgen standaard een service template.
- Module registry ondersteunt remote modules.
- Contract testing verplicht voor elke module.
- Marketplace/plugin model mogelijk maken.

10. Concrete roadmap voor jouw codebase

Eerst doen

1. Behoud Laravel als admin shell en core platform.
2. Zet productie standaard op PostgreSQL `schema` strategie.
3. Maak Taxi en Skillmatching tabellen strikt module-owned.
4. Voeg OpenAPI specs toe voor Taxi Driver API en Taxi Booking API.
5. Maak `TenantContext` expliciet in alle APIs.
6. Bouw een outbox/event laag voor Taxi, Billing en Notifications.

Daarna

1. Trek Notifications los.
2. Trek Billing/Payments los.
3. Trek Taxi Driver API los als eerste echte headless microservice.
4. Trek Matching Engine los zodra AI/embeddings belangrijker worden.
5. Trek Website Builder los wanneer tenant websites apart moeten schalen.

Niet meteen doen

- Niet direct alle controllers herschrijven.
- Niet elke module in een aparte repository forceren.
- Niet alle databases fysiek scheiden voordat service contracts stabiel zijn.
- Niet cross-service joins vervangen door synchrone API-chains voor dashboards; gebruik read models.

Minimale target voor een verkoopbare MACH-positionering

- Containerized services.
- API Gateway.
- Headless Taxi Driver API.
- Headless Website/Booking API.
- Module registry met tenant-entitlements.
- Service-owned schemas/databases.
- Event-driven notifications, billing en analytics.

11. Beslismatrix

Vraag	Advies
Moet elke module een microservice worden?	Niet altijd. Een module is een business capability. Splits pas verder als schaal, security of release-cadans dat vraagt.
Gezamenlijke database of losse database?	Target: losse DB/schema per service. Overgang: een PostgreSQL cluster met schema per module/service.
Containers of vaste DB?	Services in containers. Productiedata in managed of vaste DB, niet primair in containers.
REST of GraphQL?	REST/OpenAPI voor commands en integraties. GraphQL/BFF kan later voor admin dashboards.
Queue of event bus?	Start met outbox + queue. Groei naar RabbitMQ/NATS/Kafka wanneer volume en integraties toenemen.
Headless admin?	Gefaseerd. Eerst admin shell houden, daarna module schermen op APIs zetten.
Multi-tenant per schema of row-level?	Nu row-level met `company_id` plus module schemas. Voor enterprise klanten eventueel database-per-tenant.

12. Eindadvies

De juiste MACH-route voor NEXA is een gefaseerde composable SaaS-architectuur:

- Core blijft voorlopig de control plane.
- Modules worden eerst database- en API-afgebakend.
- Headless APIs worden leidend voor admin, websites en apps.
- Realtime en gevoelige domeinen worden als eerste losse services.
- Productie draait met containers voor services en managed databases voor state.

Deze aanpak houdt de huidige codebase bruikbaar, verlaagt rewrite-risico en maakt het platform verkoopbaar als uitbreidbare, API-first SaaS met losse modules.